TECHNICAL DEVELOPER MANUAL

Native PHP REST API Framework

Zero-Dependency Pure PHP 8+ MVC Architecture for Shared & Cloud Hosting

Table of Contents

1. System Architecture & Request Lifecycle Overview
2. Strict MVC Decomposition (Core, Models, Controllers, Middleware, Routes)
3. Database Layer & PDO Prepared Statements
4. Universal Routing, Method Detection, 404 & 405 Handling
5. Universal File & Image Upload System (User Profile Avatar Case Study)
6. Step-by-Step Guide: Adding a New Feature (Which Files to Touch)
7. External Frontend Integration (Fetch API Code Examples)
8. InfinityFree / cPanel Deployment & Security Hardening

1. System Architecture & Request Lifecycle Overview

The framework uses **pure native PHP 8+** without Composer, external packages, or command-line build steps. It is architected specifically to run smoothly on shared hosting environments (such as **InfinityFree**, **cPanel**, and local XAMPP/WAMP) where SSH access and terminal commands are unavailable.


INCOMING HTTP REQUEST
(e.g., POST /api/users, GET /api/users/1, OPTIONS)
1. Web Server (.htaccess Rewrite)
- Intercepts request; hides internal folders (app, config); rewrites URI to index.php
2. Single API Entry Point (index.php)
- Registers Native Autoloader (App\Core\Autoloader)
- Loads configurations (config/app.php, config/database.php)
- Applies Global Error/Exception Handlers (catches Throwables, returns clean JSON)
- Applies CorsMiddleware (handles allowed origins, methods, headers, and OPTIONS preflight)
- Tests/Initializes PDO Database Connection
- Automatically scans and includes all route files in app/Routes/*.php
3. Universal Router (app/Core/Router.php)
- Normalizes request path (strips base directories / subfolders automatically)
- Resolves dynamic parameters (e.g. /api/users/{id} -> extracts id = "1")
- If route matches path & method -> Dispatches to [ControllerClass, 'actionMethod']
- If route matches path but NOT method -> Sends 405 Method Not Allowed + Allow header
- If no route matches -> Sends 404 Endpoint Not Found
4. Controller Layer (app/Controllers/UserController.php)
- Receives Request object ($this->request->getBody(), $this->request->file('avatar'))
- Validates inputs (required, email, min, max, integer)
- Performs business validation (checks for email duplicates, existing records)
- Invokes Model methods (User::findById, User::create, User::update, etc.)
- Invokes Universal Uploader when processing file/image uploads
5. Model Layer (app/Models/User.php extends app/Core/Model.php)
- Executes pure SQL using PDO Prepared Statements (:name, :email, :id)
- Returns clean associative arrays or booleans (NEVER contains HTTP or presentation logic)
6. Standardized Response (app/Core/Response.php)
- Sets HTTP status code (200, 201, 400, 404, 405, 422, 500)
- Emits Content-Type: application/json; charset=UTF-8
- Outputs unified JSON payload: { "success": true/false, "message": "...", "data": ... }

2. Strict MVC Decomposition

The framework follows a strict **Model-View-Controller (MVC)** separation adapted for headless REST APIs (where the "View" is represented by standardized JSON responses):

Layer	Directory & Key Files	Responsibilities & Rules

Core Framework	`app/Core/Autoloader.php`
	`app/Core/Router.php`
	`app/Core/Request.php`
	`app/Core/Response.php`
	`app/Core/Database.php`
	`app/Core/Uploader.php`
		Foundational engine of the framework. Handles class autoloading, URL routing, HTTP request abstractions, JSON serialization, database connection pooling, and secure file uploading. Classes in this layer are reusable and framework-wide.

Models	`app/Core/Model.php` `app/Models/User.php`	**Data persistence only.** Interacts directly with the MySQL database via PDO prepared statements. Handles CRUD operations. *Strict Rule:* Models must never read `$_POST` / `$_GET` directly, and must never call `header()`, `echo`, or `exit`.
Controllers	`app/Core/Controller.php` `app/Controllers/UserController.php`	**Coordination and business logic.** Reads input via `$this->request`, validates data according to rules, calls Models, handles error conditions, and invokes `$this->success()` or `$this->error()`. *Strict Rule:* Controllers must never contain raw SQL queries.
Middleware	`app/Middleware/CorsMiddleware.php`	**Request pipeline filters.** Executes before controllers to set CORS headers (`Access-Control-Allow-Origin`, etc.) and intercept HTTP `OPTIONS` preflight requests.
Routes	`app/Routes/users.php` `app/Routes/*.php`	**Declarative mapping only.** Maps HTTP methods and URL paths to Controller actions. Contains zero business or database logic.
Configuration	`config/app.php` `config/database.php`	Centralized environment variables, database credentials, timezone, and CORS rules.

3. Database Layer & PDO Prepared Statements

Database access is managed through `App\Core\Database` and `App\Core\Model` using PHP's native **PDO (PHP Data Objects)** extension.

> **Database Singleton Pattern**
>
> `Database::getConnection()` caches the `PDO` instance statically. This guarantees that regardless of how many models are instantiated during a single HTTP request, only one physical database connection is opened to MySQL, preventing resource exhaustion on shared hosting.

Prepared Statements Protection

The base model enforces prepared statements for all database interactions. Input variables are bound as parameters rather than concatenated directly into SQL strings, completely preventing SQL Injection:

```
// In app/Core/Model.php
protected function query(string $sql, array $params = []): PDOStatement
{
    $stmt = $this->db->prepare($sql);
    $stmt->execute($params);
    return $stmt;
}

// In app/Models/User.php
public function findById(int $id): ?array
{
    $sql = "SELECT id, name, email, avatar, created_at FROM {$this->table} WHERE id = :id LIMIT 1";
    return $this->fetchOne($sql, ['id' => $id]);
}
```

4. Universal Routing, Method Detection, 404 & 405 Handling

The universal router in `app/Core/Router.php` provides expressive, declarative route registration:

```
$router->get('/api/users', [UserController::class, 'index']);
$router->post('/api/users', [UserController::class, 'store']);
$router->get('/api/users/{id}', [UserController::class, 'show']);
$router->put('/api/users/{id}', [UserController::class, 'update']);
$router->post('/api/users/{id}/avatar', [UserController::class, 'uploadAvatar']);
$router->delete('/api/users/{id}', [UserController::class, 'destroy']);
```

Dynamic Parameter Matching Mechanism

When routes are registered, patterns with placeholders like `{id}` are converted into named regular expression capture groups:

- `/api/users/{id}` → `#^/api/users/(?P<id>[^/]+)$#`
- When a request for `/api/users/42` arrives, the router matches the regex, extracts `$params['id'] = '42'`, and calls:

  ```
  call_user_func_array([$controller, $method], array_values($params)); // calls show('42')
  ```

Subfolder Auto-Detection

`Request::getPath()` inspects `$_SERVER['SCRIPT_NAME']`. If the application is hosted in a subfolder such as `http://localhost/my-api/` or `https://example.com/~username/api/`, the base directory is automatically stripped before matching against routes. No hardcoded paths are required.

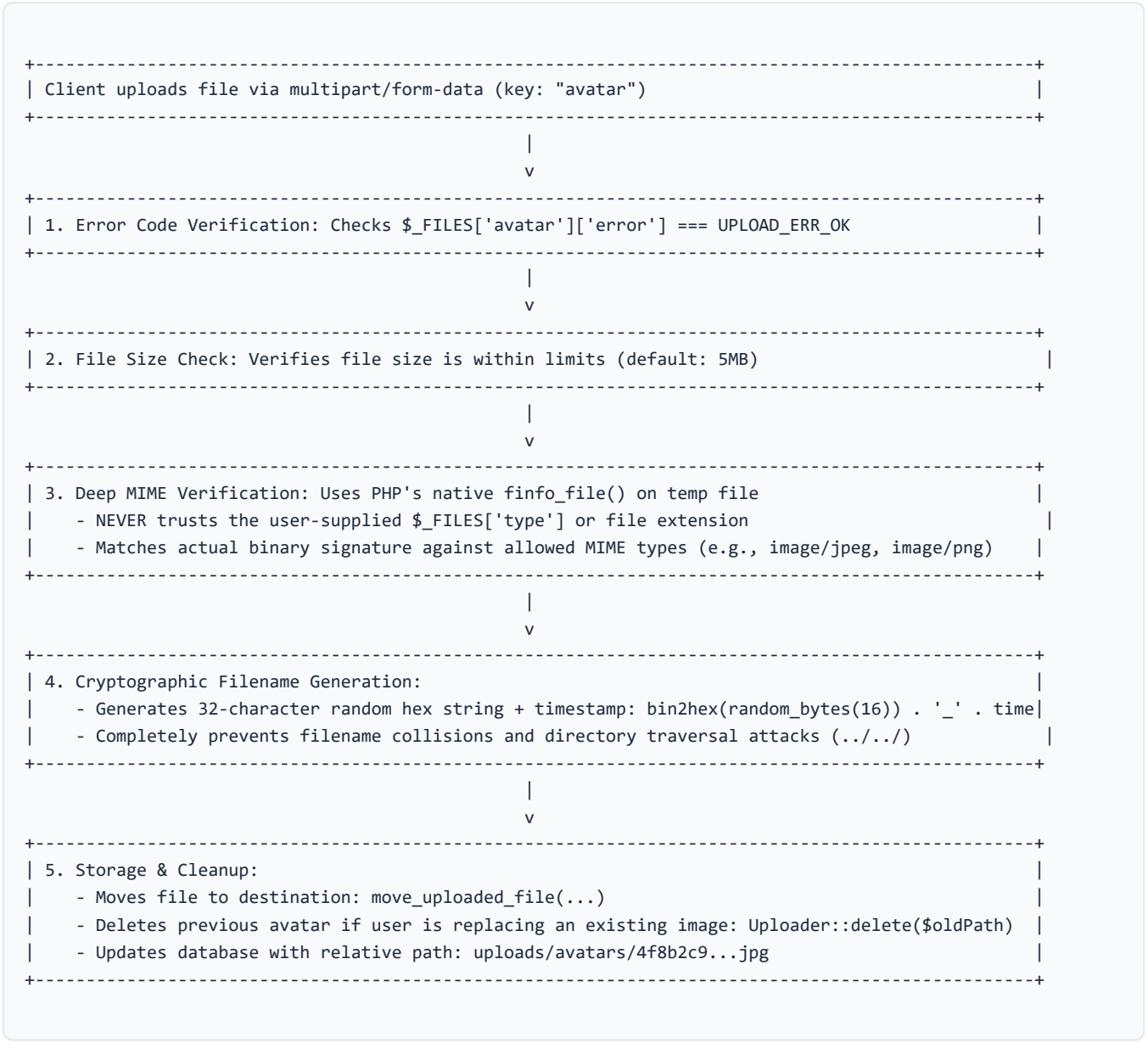
404 vs 405 Protocol Compliance

- **405 Method Not Allowed:** If a client sends a `POST` request to an endpoint registered only for `GET`, the router scans all routes. It identifies that the URL exists under other methods, sets the standard HTTP `Allow: GET, OPTIONS` header, and returns a 405 JSON response.
- **404 Endpoint Not Found:** If the requested URL matches no registered route under any HTTP method, the router returns a clean 404 JSON response.

5. Universal File & Image Upload System

The framework includes a standalone, production-ready upload engine in `app/Core/Uploader.php`. It handles file validation, secure naming, folder creation, and old file cleanup.

Upload Security Architecture



Reusable Uploader API Reference

The `Uploader` can be called from any controller for any entity (Users, Products, Documents, etc.):

```
// 1. Uploading an Image (Max 5MB)
$result = App\Core\Uploader::upload(
    $this->request->file('avatar'),     // $_FILES array
    'uploads/avatars',                  // Target directory
    App\Core\Uploader::IMAGE_TYPES,     // Allowed types: jpg, png, webp, gif
    5242880                             // 5MB limit
);

if ($result['success']) {
    $relativePath = $result['relative_path']; // e.g. "uploads/avatars/d83f...jpg"
} else {
    $errorMessage = $result['error'];
}

// 2. Deleting an Outdated File
App\Core\Uploader::delete($oldAvatarPath);
```

6. Step-by-Step Guide: Adding a New Feature (Which Files to Touch)

Because the framework follows strict MVC and automatically discovers routes, adding a new feature requires touching **exactly four files** without ever editing `index.php` or the Core files.

Example: Adding a "Products" API with Image Upload

1 Create Database Table (`database.sql`)

Create the database table in MySQL / phpMyAdmin:

```
CREATE TABLE IF NOT EXISTS products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    image VARCHAR(255) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

2 Create Model: `app/Models/Product.php`

Create the Model class. It inherits all database helpers from `App\Core\Model` :

```php
<?php

namespace App\Models;

use App\Core\Model;

class Product extends Model
{
    protected string $table = 'products';

    public function all(): array
    {
        return $this->fetchAll("SELECT * FROM {$this->table} ORDER BY id DESC");
    }

    public function findById(int $id): ?array
    {
        return $this->fetchOne("SELECT * FROM {$this->table} WHERE id = :id LIMIT 1", ['id' => $id]);
    }

    public function create(array $data): int
    {
        $sql = "INSERT INTO {$this->table} (name, price, image) VALUES (:name, :price, :image)";
        $this->execute($sql, [
            'name'  => $data['name'],
            'price' => $data['price'],
            'image' => $data['image'] ?? null,
        ]);
        return (int)$this->lastInsertId();
    }
}
```

3 Create Controller: `app/Controllers/ProductController.php`

Create the Controller class to handle requests, validate inputs, and return responses:

```php
<?php

namespace App\Controllers;

use App\Core\Controller;
use App\Core\Request;
use App\Core\Uploader;
use App\Models\Product;

class ProductController extends Controller
{
    private Product $productModel;

    public function __construct(?Request $request = null)
    {
        parent::__construct($request);
        $this->productModel = new Product();
    }

    public function index(): void
    {
        $products = $this->productModel->all();
        $this->success($products, 'Products retrieved successfully');
    }

    public function store(): void
    {
        $input = $this->request->getBody();

        // 1. Validate
        $errors = $this->validate($input, [
            'name'  => 'required|min:2|max:150',
            'price' => 'required|numeric',
        ]);

        if (!empty($errors)) {
            $this->error('Validation failed', 422, ['errors' => $errors]);
            return;
        }

        // 2. Optional Image Upload
        $imagePath = null;
        if ($this->request->hasFile('image')) {
            $upload = Uploader::upload($this->request->file('image'), 'uploads/products');
            if (!$upload['success']) {
                $this->error($upload['error'], 422);
                return;
            }
            $imagePath = $upload['relative_path'];
        }

        // 3. Save via Model
        $id = $this->productModel->create([
            'name'  => $input['name'],
            'price' => $input['price'],
            'image' => $imagePath,
        ]);

        $newProduct = $this->productModel->findById($id);
        $this->success($newProduct, 'Product created successfully', 201);
    }
}
```

4 Create Route File: `app/Routes/products.php`

Create a new route file in `app/Routes/`:

```
<?php

use App\Controllers\ProductController;

/** @var \App\Core\Router $router */
$router->get('/api/products', [ProductController::class, 'index']);
$router->post('/api/products', [ProductController::class, 'store']);
```

> **Automatic Route Activation**
>
> Because `index.php` automatically scans `app/Routes/*.php` using `glob()`, your new endpoints (`GET /api/products` and `POST /api/products`) are immediately active! You do not need to edit `index.php` or restart any service.

7. External Frontend Integration (JavaScript Fetch API)

Below are clean JavaScript examples demonstrating how external web applications (React, Vue, Angular, or Vanilla JS) communicate with the API:

1. GET All Users

```
const response = await fetch('http://localhost/my-api/api/users');
const result = await response.json();
console.log(result.data); // Array of user objects
```

2. GET Single User by ID

```
const response = await fetch('http://localhost/my-api/api/users/1');
const result = await response.json();
if (response.ok) {
    console.log(result.data); // User object
} else {
    console.error(result.message); // e.g. "User not found"
}
```

3. POST Create User (JSON Payload)

```
const response = await fetch('http://localhost/my-api/api/users', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json'
    },
    body: JSON.stringify({
        name: 'Sarah Connor',
        email: 'sarah@example.com'
    })
});
const result = await response.json();
console.log('Created User ID:', result.data.id);
```

4. PUT Update User Profile

```
const response = await fetch('http://localhost/my-api/api/users/1', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        name: 'Sarah Connor-Reese'
    })
});
const result = await response.json();
console.log('Updated:', result.data);
```

5. POST Upload Avatar Image (Multipart FormData)

```
// Note: Do NOT set 'Content-Type': 'multipart/form-data' header manually.
// The browser sets it automatically with the correct multipart boundary.
const fileInputElement = document.querySelector('#avatarInput');
const formData = new FormData();
formData.append('avatar', fileInputElement.files[0]);

const response = await fetch('http://localhost/my-api/api/users/1/avatar', {
    method: 'POST',
    body: formData
});
const result = await response.json();
console.log('New Avatar URL:', result.data.upload.url);
```

6. DELETE User

```
const response = await fetch('http://localhost/my-api/api/users/1', {
    method: 'DELETE'
});
const result = await response.json();
console.log('Deleted status:', result.success);
```

8. InfinityFree / cPanel Deployment & Security Hardening

Deployment Step-by-Step

Step	Action in cPanel / InfinityFree	Notes
1. Create Database	Open **MySQL Databases** in vPanel. Create a new database name.	Note the MySQL Hostname (e.g. `sql300.infinityfree.com`), Database Name, Username, and Password.
2. Import Schema	Open **phpMyAdmin**, select your database, click **Import**, and upload `database.sql`.	Creates the `users` table with proper constraints and indexes.
3. Configure Credentials	Open `config/database.php` and fill in your remote MySQL credentials.	Set `'debug' => false` in `config/app.php` for production environments.
4. Upload Files	Using **cPanel File Manager** or FTP (FileZilla), upload the entire folder into `htdocs/` (or `public_html/`).	Ensure hidden files like `.htaccess` are transferred.

Security Hardening Features Included

- **Sensitive Directory Shielding:** In the root `.htaccess`, direct HTTP access to internal PHP directories (`app/` and `config/`) is blocked with an HTTP 403 Forbidden rule:

```
RewriteRule ^(app|config)/.*$ - [F,L]
```

- **Upload Folder Execution Lock:** The `uploads/.htaccess` file completely forbids the execution of PHP or script files inside the uploads directory:

```
<FilesMatch "(?i)\.(php|phtml|php3|php4|php5|php7|php8|phps|cgi|pl|exe)$">
    Require all denied
</FilesMatch>
Options -Indexes -ExecCGI
```

  Even if an attacker bypasses application-level MIME checks, Apache refuses to execute any uploaded script.
- **Information Disclosure Shielding:** When `'debug' => false` in `config/app.php`, database errors and internal exceptions log silently to the server error log, presenting a clean `"Internal Server Error"` response to API consumers with zero exposed credentials or stack traces.
- **Authorization Header FastCGI Fix:** Apache shared hosting commonly strips the HTTP `Authorization` header before passing requests to PHP. The root `.htaccess` automatically forwards the authorization header into the PHP environment:

```
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
```